{EPITECH}

SUMMER SCHOOL_

< PART 4 PROJECT - THE LOCAL TUNNEL PIPELINE (NGROK) />

SUMMER SCHOOL

< AI & AUTOMATION: BRIDGING CLOUD AND LOCALHOST />

> ”Data is a precious thing and will last longer than the systems themselves.”
> — **Tim Berners-Lee**

Introduction

In the previous module, you successfully connected Google Forms to Gmail and learned how to extract clean text from raw JSON payloads. However, your data is still confined entirely inside Google's cloud ecosystem.

Now that you have built your own local **Node/Express Back-End API** in your Web Development classes, the ultimate goal is to feed incoming form submissions directly into your own server code!

But here is the engineering catch: Your Express server is currently running on `http://localhost:3000`. Since `localhost` is completely isolated inside your private machine, Make.com (which lives on the cloud) cannot see or send an HTTP POST request to your computer.

The Case

Your School Digital Newspaper is finally ready to go live! The Web Development team has finished the React frontend, which displays articles by reading a local database (`articles.json`) managed by an Express Back-End.

But there is a major problem: Right now, your automated editorial pipeline only saves articles in a Google Doc. The frontend website cannot read a Google Doc! If you leave it like this, the website will remain empty forever.

To actually publish the news, the editorial workflow must push the approved drafts directly into the Web team's local backend API. Your mission is to build the final bridge that connects the cloud to your local code, allowing for instant publication on the real website.

The Theory: What is a Local Tunnel?

When you run an API locally, your computer acts as a hidden server. To allow external cloud applications (like Make, Stripe, or Discord bots) to send Webhooks or HTTP requests to your machine, you need a bridge.

Ngrok is a tunneling software that creates a secure, temporary public URL (e.g., `https://xyz.ngrok-free.app`) that points directly to your local port (e.g., `3000`). When Make.com triggers and hits that public Ngrok URL, Ngrok instantly forwards the entire payload down into your local machine's Express server.

The Mission

You must upgrade your Make.com scenario to route data directly into your local database file (`articles.json`) via your Express API, using Ngrok as the communication bridge.

Your technical setup must follow these strict requirements:

1. Expose your Server (The Ngrok Setup)

✓ Download and install **Ngrok** on your machine.
✓ Start your local Express server (`node server.js` or `npm run dev`) on port 3000.
✓ Open a new, separate terminal and fire up the tunnel using the following command:

```
ngrok http 3000
```

✓ Copy the secure forwarding URL generated by Ngrok (the one starting with `https://`). Keep this terminal open!

2. The Automation Update (Make.com -> HTTP Module)

✓ Open your Make.com scenario from Part 3.
✓ Add a brand new **HTTP** module and choose the action **"Make a request"**.
✓ **Configure the HTTP Node:**
  – **URL:** Paste your public Ngrok URL and append your endpoint path (e.g., `https://your-id.ngrok-free.app/api/articles`).
  – **Method:** Select `POST`.
  – **Body type:** Select `Raw`.
  – **Content type:** Select `JSON (application/json)`.
  – **Request content:** Map the variables from your Google Form into a clean, structured JSON format matching your Express expected fields:

```
{
  "title": "[Map Article Title]",
  "excerpt": "[Map Article Draft]",
  "author": "[Map Journalist Name]"
}
```

3. The Local Engine (Express Verification)

✓ When a user submits the Google Form, Make.com must catch the submission, format the JSON, and shoot it to your Ngrok link.
✓ Your local Express server must catch this payload on your `POST /api/articles` route, assign it a unique ID, and save it inside your local `articles.json` file automatically.

Deliverables

At the end of this Project, you will learn how to build a fully automated editorial pipeline that connects cloud services to your local server.

1. **The Ngrok Tunnel:** Your ngrok tunnel is connected and running, and you can see the public URL in your terminal.
2. **The Live Loop Check:** Open your Google Form, fill in a brand new mock article, and hit submit, it must trigger the Make.com scenario and fire the HTTP request to your local Express server.
3. **The Database Proof:** Open your local code editor and show your reviewer your `articles.json` file. The article you just submitted via the Google Form must be sitting inside the file, cleanly updated without any manual interaction!

v1